

Rapport Programmation Web

Clément Bedour numéro étudiant : 4100372 URL du site : <https://etud.insa-toulouse.fr/~bedour/>

Ce rapport retrace les principales fonctions / processus de création du jeu 'Wars Star' (toutes ressemblances à une marque est fortuite). Ce projet a été fait sur VS Code et une utilisation partielle de GitHub.

Fonctionnalités implémentées

Affichage / Cohérence visuelle :

Pour la page d'accueil et le classement je trouvais plus simple de faire une grille pour le visuel. Elle est sur le index.html ligne 17 et dimensionnée sur style_intro.css ligne 15. Je me suis aidé d'un site pour m'aider avec le dimensionnement et le code. (<https://cssgridgenerator.io/>).

J'ai aussi créé plusieurs boutons pour diriger le joueur d'une page HTML à une autre, exemple ligne 41 dans index.html. J'ai pu donc tous leur donner la même apparence grâce au CSS ligne 92 dans style_intro.css.

Pour donner un pseudo au joueur j'ai utilisé une entrée textuelle en limitant les caractères pouvant être utilisés ligne 32 dans index.html. J'ai donc fait un message d'erreur si le pseudo n'est pas valide sur script_intro.js avec la condition ligne 40 et 45. Ce pseudo est utilisé dans classement.html ligne 16. Si aucun pseudo n'est rentré ou qu'il n'est pas bon, il affichera un pseudo informatif grâce à la condition ligne 54 de script_intro.js

Création du jeu

Toutes les lignes relatives à un fichier JS seront pour le script_facile.js. Il y a les mêmes lignes présentes dans les fichiers script_difficile.js et script_extreme.js. Elles seront adaptées aux fichiers car il y a plus de blocks et des mouvements différents. L'image du fond est aussi légèrement différente.

1. Visuel des blocks

La grille de jeu est faite ligne 22 de jeu_facile.html et les blocks dans la même division ligne 23,24,25 du même fichier. Le 'hero' est aussi un block mais il va utiliser d'une manière totalement différente. Pour aider à la maintenance du jeu et une cohérence dans le code j'ai regroupé le plus possible les attributs identiques pour les blocks (exemple ligne 81 de style_jeu.css).

Pour voir les différents visuels ils sont dans style_jeu.css

- Les grilles des jeux ligne 44 et les spécialités pour chaque niveau ligne 56, 61 et 66
- Le 'hero' est le même pour tous, ligne 72 et 81
- Les blocks pour tous les niveaux sont ligne 81
- Pour les différents visuels des blocks, lignes 95, 101, 107 et 113

2. Vitesse et apparition des blocks

J'ai utilisé différentes vitesses pour les blocks lignes 140, 150, 163, 175 et 187 de style_jeu.css. L'attribution de cette vitesse sur les blocks se fait lignes 91, 119, 123, 127, 131 et 136. Pour le niveau extrême, je voulais que les blocks ne commencent pas tous aux mêmes endroits. Le fait de mettre une valeur négative ne m'a pas été intuitive et j'ai beaucoup cherché pour trouver une solution.

Pour savoir où les blocks devaient apparaître j'ai choisi d'utiliser un tableau. Je trouvais cette utilisation plus simple en me disant que chaque composante du tableau était le premier pixel de la colonne ligne 73 du JS. Le premier block apparaît sur une colonne aléatoirement grâce à la ligne 75 du JS. Puis je fais apparaître le second blocks 50% du temps sur une autre colonne si ce n'est pas la même que le block précédent, ligne 83 et 85 du JS. Le premier block est affiché grâce à la commande ligne 76.

Pour les niveaux suivants, j'ai supprimé l'apparition à 50%. Ils apparaissent tous seulement s'ils ne sont pas dans une colonne déjà utilisée ligne 81, 88, 97 de script_difficile.js. Je devais faire disparaître les blocks car les colonnes changent à chaque exécution. Je les cache en les mettant à -220px, donc en dehors de la grille de jeu.

3. Les touches

La reconnaissance des touches est faite grâce à la fonction ligne 52 du JS et le mouvement du 'hero' est fait avec la fonction ligne 16 et 26. Il fallait vérifier où le 'hero' était et le faire aller du côté opposé s'il était sur une bordure.

4. Classement

J'ai fait un tableau par niveau de difficulté pour le classement, je trouvais ça plus simple et facile pour le trier et retrouver le premier élément (HighScore). Avant de créer un classement il fallait créer un score qui s'incrémente toutes les secondes. Ce score a été fait simplement avec une fonction ligne 123 du JS, sachant que cette boucle a une limite de 1000 millisecondes, ça revient à le faire toutes les secondes.

Après avoir mon score je me suis attaqué au classement (présent dans script_intro.js). J'ai créé une fonction ligne 39 qui :

- Récupère le score actuel (current)
- Récupère le tableau avec l'id correspondant au niveau de difficulté ou en crée un vide s'il n'existe pas ligne 40
- Mets le score courant dans la liste grâce à push
- Trie la liste avec sort, le paramètre 'b-a' me permet d'avoir une liste triée par ordre décroissant et réutilisable pour l'affichage du top5
- Je garde mes 5 derniers éléments (utile pour l'affichage plus tard) avec slice(0,5)
- Je sauvegarde ma ligne
- Puis j'affiche le 1^{er} élément car c'est le plus grand

Pour afficher le top 5 dans classement.html, j'ai créé une grille pour un visuel plus agréable (avec le même site que la première fois). Puis j'ai créé une division pour le titre et 3 autres pour afficher les tableaux liés au classement pour chaque niveau.

J'ai créé une fonction dans script_intro.js ligne 5 qui crée une division, affiche chaque élément du tableau s'il n'est pas nul ligne 14 ou 'Aucun score' s'il n'y a pas 5 parties qui ont été jouées. Sinon il marque 'Aucun partie jouée' c'est le cas ligne 23. Grâce à cette fonction générique, je l'appelle 3 fois, ligne 33,34,35 pour mes 3 niveaux de difficulté et ça m'affichera les valeurs dans chaque division.

5. Popup

La fenêtre de fin de jeu m'a posé beaucoup de problèmes. J'ai eu du mal à créer la fenêtre à partir d'un JS, car il fallait mélanger du HTML avec du JS et sans message d'erreur c'était assez complexe. Avant de faire apparaître la fenêtre je dois mettre tous les blocks et le timer en pause, effectuer avec la fonction ligne 151 de script_facile.js. Il fallait créer une seule fois les divisions qui allaient faire apparaître un peu de texte et le bouton rejouer. Pour le bouton rejouer il suffisait juste d'actualiser la page en cours, ligne 184.

Conclusion

J'ai beaucoup aimé faire ce projet, je me suis beaucoup aidé des ressources disponibles sur internet (une grande partie de mes sources sont disponibles dans le README). J'ai même fini par avoir beaucoup de ressources directement dans mon feed Instagram, ce qui m'a donné des contenus pertinents, le bouton télécharger, le fait de mixer les images (ligne 9 jeu_difficile.html et ligne 32 style_jeu.css) pour faire un peu varier le fond en fonction de la difficulté, et d'autres que j'oublie.

Les points à améliorer sont : Des bugs que je n'ai pas réussi à corriger comme, si on fait un 'alt tab, aucune collision n'est prise en compte. Lors du premier cycle, tous les blocks sont dans la même colonne. Je ne m'en suis rendu compte en parlant avec des camarades en 4^{ème} année. Je ne devrais pas mettre des tailles fixes 'px' dans le CSS. J'aurais dû utiliser % ou en équivalent vw et vh.